

Contents

Contents	1
Document Control	1
Executive Summary	2
Project Overview	3
Business Problem	3
Business Goals	4
Project Objectives	4
Stakeholders	4
User Personas	5
Project Scope	5
Functional Requirements	6
Non-Functional Requirements	8
User Stories	9
Business Rules	9
Assumptions	10
Constraints	10
Risks	10
Success Criteria	12
Future Roadmap	12
Glossary	12

Project Requirements Document IPO Allotment Verification System Prepared by: Bhavya Bularani Version 1.2 · July 2026

Contents

- Document Control
- Executive Summary
- Project Overview
- Business Problem
- Business Goals
- Project Objectives
- Stakeholders
- User Personas
- Project Scope
- Functional Requirements
- Non-Functional Requirements
- User Stories
- Business Rules
- Assumptions
- Constraints
- Risks
- Success Criteria
- Future Roadmap
- Glossary

Document Control

Item	Details
Document Name	Project Requirements Document (PRD)
Project	IPO Allotment Verification System

Item	Details
Version	1.2
Status	Draft
Author	Bhavya Bulani
Reviewers	Project Supervisor, Technical Reviewer
Last Updated	July 2026

Version History

Version	Date	Description	Author
1.0	July 2026	Initial draft compiled from project discovery discussions	Bhavya Bulani
1.1	July 2026	Added Automatic IPO Detection, Multi-IPO Selection, and Result Caching to in-scope requirements	Bhavya Bulani
1.2	July 2026	Switched primary database from SQLite to MySQL; updated technology stack, assumptions, and future roadmap accordingly	Bhavya Bulani

Approval Matrix

Role	Name	Status
Project Supervisor	Pending	Pending
Technical Reviewer	Pending	Pending

Executive Summary

The IPO Allotment Verification System is a web-based internal tool designed to automate the process of checking IPO (Initial Public Offering) allotment status for brokerage clients. Today, this process is handled manually: an operator visits one of several registrar websites, enters a client's PAN or Client Code one at a time, and records the result. For a brokerage with hundreds or thousands of active clients per IPO, this is slow, repetitive, and error-prone.

The system replaces this manual workflow with a single application that accepts either one client (typed in directly) or many clients at once (uploaded as an Excel file), checks each client's allotment status across multiple registrar websites with automatic fallback if one site is unavailable, and returns a consolidated result — viewable on screen and downloadable as an Excel report.

Version 1.1 extended this with three capabilities pulled forward from the future roadmap: automatic detection of newly announced IPOs, multi-IPO selection within a single run, and

result caching. **Version 1.2 changes the underlying database from SQLite to MySQL**, reflecting the concurrent, multi-worker write pattern the system produces once parallel processing and multi-IPO fan-out are both active — this is an infrastructure change, not a change to any user-facing capability.

The guiding principle behind the design remains that reliability, not the interface, is the hard part of this project. Registrar websites are not built for automation, so the system must be resilient to slow responses, downtime, and inconsistent input formats, and must degrade gracefully rather than fail silently.

Project Overview

Field	Value
Project Name	IPO Allotment Verification System
Project Type	Internal / Enterprise Web Application
Domain	Financial Technology (FinTech) — Brokerage Operations
Target Users	Brokerage firms, brokerage staff, relationship managers, internal operations teams

Proposed Technology Stack

Layer	Technology
Frontend	React + Tailwind CSS
Backend	FastAPI (Python)
Automation Engine	Playwright (preferred) or Selenium, for registrars without a public API
Excel Processing	pandas + openpyxl
Concurrency / Queue	asyncio-based worker pool
Result Cache	In-process / lightweight cache layer (in-memory with TTL, backed by MySQL)
Database	MySQL (InnoDB), from the initial version
ORM / Migrations	SQLAlchemy + Alembic
IPO Master Data	Upstox IPO API (or equivalent public data source), polled for new-IPO detection

Business Problem

IPO allotment verification is currently a repetitive, manual process for brokerage firms. A typical operator workflow looks like this:

1. Identify the IPO in question.
2. Determine which registrar (Link Intime, KFin Technologies, Bigshare, MUFG Intime, etc.) is handling that IPO.
3. Open that registrar's website.
4. Enter the client's PAN or Client Code.
5. Read and record the allotment result.
6. Repeat the process for every client, one at a time — and again for every additional IPO the client applied to.

This creates several concrete problems:

- Multiple registrar websites, each with a different interface and input format.
- Manual data entry errors when transcribing PAN numbers or client codes.
- Time cost that scales linearly with client count.
- No fallback when a registrar's website is slow or temporarily down.
- No consolidated record of what was checked, when, and with what result.
- Operators must manually notice and add each newly announced IPO before it can be checked.
- Clients who applied to multiple IPOs require a fully separate run per IPO.
- The same client/IPO combination is often checked more than once in a short window, sending duplicate, avoidable requests to registrar websites.

Business Goals

- Reduce manual effort required to verify IPO allotment status.
- Centralize IPO application data for all clients in one place.
- Provide a single, consistent search interface regardless of which registrar is involved.
- Improve operational turnaround time from hours to minutes for large client batches.
- Keep the IPO list current automatically.
- Let operators check a client list against multiple IPOs in one pass.
- Reduce unnecessary load on registrar websites via caching.
- Build a foundation — including a database that supports real concurrent write load — that can scale to support additional registrars, additional IPOs, and eventually multiple brokerage firms.

Project Objectives

- Allow a single client to be checked by entering a PAN or Client Code directly.
- Allow bulk checking by uploading an Excel file (.xlsx / .xls) containing client details.
- Validate all input before it is sent to any registrar.
- Check allotment status across at least four registrar websites, falling back automatically.
- Run checks for multiple clients in parallel, backed by a database (MySQL) that safely supports concurrent writes from many workers at once.
- Automatically detect and add newly announced IPOs from the configured data source.
- Allow a user to select one or more IPOs for a single run.
- Cache recent results per client/IPO combination and serve cached results within the TTL window.
- Handle registrar CAPTCHA challenges gracefully — automatic where possible, operator-assisted as the standard fallback, and an optional disabled-by-default third-party solver.
- Present results with clear status categories, and clearly indicate when a result was served from cache.
- Generate a downloadable Excel report of results, and maintain a history of past checks.
- Log every check run to support troubleshooting.

Stakeholders

Stakeholder	Responsibility
Project Sponsor / Supervisor	Approves project direction and scope
Development Team	Designs and builds the application
Brokerage Staff	Day-to-day users who run allotment checks
Operations Team	Prepares and uploads client Excel files, manages data quality

Stakeholder	Responsibility
Clients (end beneficiaries)	Receive faster, more accurate allotment confirmation

User Personas

Persona 1 — Brokerage Staff (Primary User)

Goals — Check allotment status for one client quickly. Upload a client list and get a complete report. Check a client list against several IPOs at once. Trust that the result shown is accurate and current.

Pain Points — Repeated manual searches on multiple registrar sites. No way to check hundreds of clients without spending hours on it. Data entry mistakes. Having to manually notice and add newly announced IPOs.

Persona 2 — Operations Executive

Goals — Manage large Excel uploads reliably. Understand exactly which rows failed validation and why. Track the history of past uploads. Avoid generating unnecessary registrar traffic on repeat checks.

Pain Points — Inconsistent column naming across source files. No visibility into progress during a long-running bulk check. No audit trail. Repeating an entire multi-IPO check manually, one IPO at a time.

Project Scope

In Scope (Version 1.2)

- Single-client check via direct PAN / Client Code entry.
- Bulk check via Excel upload (up to 10,000 rows per file).
- Automatic detection of relevant columns in uploaded files.
- Input validation for PAN and Client Code formats prior to any external lookup.
- Automatic detection of newly announced IPOs.
- IPO selection supporting one or more IPOs per run.
- Multi-registrar fallback checking (Link Intime, KFin Technologies, Bigshare, MUFG Intime).
- Parallel processing of bulk checks using a configurable worker pool, backed by MySQL for safe concurrent writes.
- Result caching, backed by MySQL.
- Real-time progress display during bulk runs, across all selected IPOs.
- Result categorization with a clear cache-vs-fresh indicator.
- Excel export of results.
- Summary dashboard, broken down by IPO when multiple IPOs are selected.
- Search within a completed result set.
- History of past check runs.
- Completion notifications.
- Run-level logging.

Out of Scope (Version 1.2)

- Online IPO application submission.
- Payment gateway integration.

- Trading functionality or portfolio management.
- Native mobile application.
- Multi-broker / multi-tenant administration.
- Full authentication and role-based access control (planned as a future enhancement — see Future Roadmap).
- Email / WhatsApp notifications to clients.
- Public REST API for other internal systems.

Functional Requirements

ID	Requirement	Priority
FR-001	The system shall allow a user to enter a single PAN or Client Code and check its allotment status.	High
FR-002	The system shall allow a user to upload an Excel file (.xlsx/.xls) containing multiple clients.	High
FR-003	The system shall auto-detect relevant columns in an uploaded file regardless of header naming variations.	High
FR-004	The system shall validate PAN format (10 characters: 5 letters, 4 digits, 1 letter) before submitting it to any registrar.	High
FR-005	The system shall validate Client Code format according to the applicable broker format before submission.	High
FR-006	The system shall reject or flag rows that fail validation without sending them to any external website.	High
FR-007	The system shall allow the user to select the relevant IPO(s) from a dropdown/multi-select list.	High
FR-008	The system shall check allotment status against Registrar 1 (e.g., Link Intime) first.	High
FR-009	The system shall automatically fall back to Registrar 2, 3, and 4 in sequence if the prior registrar fails, times out, or is unreachable.	High

ID	Requirement	Priority
FR-010	The system shall apply a short timeout (e.g., ~3 seconds) per registrar attempt.	Medium
FR-011	The system shall process multiple clients concurrently using a worker pool, with database writes handled safely under MySQL's concurrent-write model.	High
FR-012	The system shall display a live progress indicator during bulk checks.	Medium
FR-013	The system shall categorize each result as one of: Allotted, Not Allotted, Website Error, Invalid PAN, Timeout, Server Busy.	High
FR-014	The system shall generate a downloadable Excel report containing Name, PAN/Client Code, IPO, Status, Registrar checked (or Cache), and timestamp.	High
FR-015	The system shall display a summary dashboard after a run completes, broken down per IPO when multiple IPOs were selected.	Medium
FR-016	The system shall allow the user to search within a completed result set by name or identifier.	Medium
FR-017	The system shall store a history of past check runs for later retrieval.	Medium
FR-018	The system shall notify the user when a bulk check completes.	Low
FR-019	The system shall maintain a run-level log capturing start time, end time, registrar(s) used, timeout count, success count, failure count, and cache-hit count.	Medium
FR-020	The system shall reject Excel uploads exceeding 10,000 rows or an unsupported file type.	High

ID	Requirement	Priority
FR-021	The system shall automatically sync newly announced IPOs from the configured IPO data source on a scheduled and/or manually triggered basis.	High
FR-022	The system shall allow a user to select multiple IPOs for a single check run and shall check every client in the run against each selected IPO.	High
FR-023	The system shall check for a valid cached result for a given client/IPO combination before dispatching a new registrar request.	High
FR-024	The system shall expire cached results after a configurable TTL and re-check against registrars once expired.	Medium
FR-025	The system shall detect when a registrar presents a CAPTCHA challenge and, by default, pause that client's check and prompt an operator to solve it manually via the UI (Assisted Mode).	High
FR-026	The system shall support an optional, administrator-enabled CAPTCHA-solving provider mode (Solver API) behind a common provider interface, disabled by default.	Low
FR-027	The system shall persist all operational data — clients, IPOs, results, batches, logs — in a MySQL database accessed through a connection pool sized to the configured worker concurrency.	High

Non-Functional Requirements

Category	Requirement
Performance	A bulk run of ~1,000 clients should complete in approximately 2-3 minutes using parallel workers. Cache hits should return near-instantly.
Reliability	The system must continue operating and produce a usable partial result even if one or more registrar websites are down.
Usability	Column detection must tolerate common header naming variations. Multi-IPO selection must remain clear even with several IPOs chosen.
Scalability	The architecture should support adding new registrars as independent modules, and MySQL should comfortably support growing client/result volume without an engine change.
Auditability	Every run must be logged with enough detail to diagnose failures after the fact.
Data Handling	PAN numbers and client data are sensitive; they must be handled and stored with appropriate care, including secure MySQL credential management (see the Security & Access Control Document).
Compliance	Automated interaction with registrar websites must respect each site's terms of use and avoid excessive request rates.

User Stories

- As a brokerage staff member, I want to enter a single client's PAN and immediately see their allotment status.
- As a brokerage staff member, I want to upload an Excel file of clients, so that I don't have to check each one manually.
- As an operations executive, I want the system to tell me which rows in my upload were invalid and why.
- As a user, I want the system to automatically try another registrar website if the first one fails.
- As a user, I want to see progress while a large batch is being checked.
- As a user, I want newly announced IPOs to appear automatically in the selector.
- As a user, I want to select more than one IPO at once.
- As a user, I want the system to reuse a recent result instead of re-checking the registrar.
- As a user, I want to clearly see when a result came from cache versus a fresh check.
- As a user, I want to download the results as an Excel file.
- As an operations executive, I want to see a summary dashboard after each run.
- As a user, I want to search within my results.
- As a user, I want to see my past check runs.
- As an operations executive, I want a log of each run.

Business Rules

- No unvalidated PAN or Client Code may be sent to a registrar website.

- A registrar is only marked “failed” for a given client after its configured timeout/attempt limit is reached.
- If all configured registrars fail for a client, the result is recorded as an error state, not silently dropped.
- Every completed run must be saved to history.
- Excel uploads are capped at 10,000 rows.
- A newly detected IPO must be validated before it is made selectable to users.
- When multiple IPOs are selected, each client/IPO pair is treated as an independent check.
- A cached result is only served if it has not exceeded its configured TTL.
- Every result record must indicate its source (registrar name or “Cache”).
- All writes to the database must go through the application’s data-access layer (SQLAlchemy models), never raw, ad hoc SQL against MySQL.

Assumptions

- Registrar websites remain reachable over the public internet and do not require credentials to query public allotment status.
- The structure of registrar result pages is stable enough that automated parsing will not need to change on every run.
- Users have basic familiarity with Excel for preparing upload files.
- IPO master data can be sourced from a data provider such as the Upstox IPO API, reliably enough to support automatic detection.
- Allotment results are reasonably stable within a short window, making short-TTL caching safe.
- Some registrars may occasionally present CAPTCHA challenges; manual/assisted resolution is assumed sufficient for expected volumes.
- A MySQL server (local, containerized, or managed) will be available in every environment the application runs in — development, staging, and production — since it is now a hard dependency rather than an optional scale-up path.

Constraints

- Registrar websites are third-party systems outside the project’s control.
- Version 1.2 does not include authentication/authorization, so it should initially be deployed in a trusted internal environment only.
- Browser automation (Playwright) is inherently slower and more fragile than a direct API integration.
- The IPO data source used for automatic detection may itself have rate limits or availability constraints.
- MySQL must be provisioned and reachable (host, port, credentials) in every environment before the application can start; there is no embedded-database fallback as there was under the earlier SQLite plan.

Risks

Risk	Impact	Likelihood	Mitigation
Registrar website changes its page layout	High	Medium	Modular, per-registrar scraping components that can be updated independently

Risk	Impact	Likelihood	Mitigation
Registrar blocks automated requests	High	Medium	Conservative request pacing, fallback across multiple registrars, manual-check fallback path Row cap (10,000), background processing, worker pool tuning Data-handling controls defined in the Security & Access Control Document Flexible column auto-detection and clear validation error reporting Validation of synced IPO records before selectable; manual override/edit capability Per-registrar rate limiting applies across the whole run; caching reduces duplicate load Conservative, configurable TTL; clear “served from cache” labeling Assisted (manual-solve) mode as standard fallback; optional Solver API Properly sized connection pool, short-lived connections per write, monitoring of connection saturation Docker Compose setup for local development to keep MySQL provisioning simple for a single developer
Large Excel uploads slow down the system	Medium	Low	
Sensitive PAN data mishandled	High	Low	
Users upload inconsistent file formats	Medium	Medium	
IPO auto-detection source returns incomplete or incorrect IPO data	Medium	Medium	
Multi-IPO runs multiply registrar request volume	Medium	Medium	
Stale cached results shown to users	Medium	Low	
A registrar introduces or changes CAPTCHA challenges	Medium	Medium	
MySQL becomes a single point of failure / connection bottleneck under concurrent parallel workers	Medium	Medium	
Environment/deployment friction from requiring a provisioned MySQL server everywhere (vs. SQLite’s zero-setup model)	Low	Medium	

Success Criteria

- A batch of 1,000 clients can be checked in under 5 minutes end-to-end.
- Fallback across registrars functions correctly when a registrar is deliberately made unreachable in testing.
- Column auto-detection correctly identifies PAN/Client Code columns across the naming variations listed in this document.
- Users can retrieve any past run's results from history without re-running the check.
- No invalid PAN/Client Code is ever transmitted to a registrar website.
- A newly announced IPO becomes selectable in the UI without any manual data entry.
- A client list can be checked against at least three IPOs in a single run.
- A repeat check for the same client/IPO combination within the cache TTL is served from cache.
- The application performs correctly under concurrent parallel-worker writes to MySQL, with no lost or corrupted result rows during a large bulk run.

Future Roadmap

- Email or WhatsApp notifications to clients with their individual allotment status.
- User login with role-based access control (admin / operator) — see the Security & Access Control Document for the target access-control model.
- A REST API allowing other internal systems to trigger allotment checks programmatically.
- Retry with exponential backoff for transient registrar failures.
- Audit logs identifying which user ran each check and when.
- Read replicas / horizontal scaling of MySQL and eventual multi-broker support, as usage grows.

Glossary

Term	Definition
PAN	Permanent Account Number — a 10-character alphanumeric tax ID issued to individuals/entities in India.
Client Code	Broker-assigned identifier for a client account, format varies by broker.
Registrar	An organization responsible for processing IPO applications and allotment results on behalf of the issuer.
Allotment Status	Whether a client's IPO application was allotted shares or not.
BOID / DP ID	Beneficiary Owner ID / Depository Participant ID.
Fallback	The process of trying an alternate registrar/website when the primary one fails or times out.
Auto-Detection	The automated process of syncing newly announced IPOs from an external data source.
Result Cache	A short-lived store of recent check results, used to avoid duplicate registrar requests.
TTL (Time-to-Live)	The configured duration for which a cached result is considered valid.

Term	Definition
InnoDB	MySQL's default transactional storage engine, used here for foreign-key enforcement and row-level locking under concurrent writes.
Connection Pool	A managed set of reusable database connections shared across the application's workers, avoiding the overhead of opening a new MySQL connection per request.